

Creating a One-Dimensional NumPy Array (1 to 10)

Python Program with Algorithm, Source Code and Output

Aim

To create a one-dimensional NumPy array containing the numbers 1 to 10 using two methods (`np.array()` and `np.arange()`), inspect its properties, and demonstrate vectorized operations, indexing and slicing.

Algorithm

Step 1: START.

Step 2: Import the NumPy library as `np`.

Step 3: Method 1: Create array `arr1` from the list `[1, 2, ..., 10]` using `np.array()`.

Step 4: Method 2: Create array `arr2` using `np.arange(1, 11)` (stop value 11 is excluded, so 10 is included).

Step 5: For each array, display its contents, data type (`dtype`), shape, number of dimensions (`ndim`) and size.

Step 6: Compare `arr1` and `arr2` using `np.array_equal()` and print the result.

Step 7: Apply vectorized operations on `arr1`: multiply by 2, add 100, compute sum and mean.

Step 8: Demonstrate indexing and slicing: first element `[0]`, last element `[-1]`, first five values `[ :5]`, every other value `[ : :2]`.

Step 9: STOP.

Source Code

```
"""
=====
CREATING A ONE-DIMENSIONAL NUMPY ARRAY (THE NUMBERS 1 TO 10)
=====
IN PLAIN ENGLISH:
    A NumPy array is like a Python list, but built specifically for
    fast, whole-array math. A "one-dimensional" array is simply a
    single row of numbers - no rows-and-columns grid, just one
    straight line of values, like beads on a single string.

    This script creates that array two different ways, inspects its
    basic properties, and shows a few quick examples of why NumPy
    arrays are so useful compared to plain Python lists.
=====
"""

import numpy as np

# =====
# 1. CREATE THE ARRAY - TWO COMMON WAYS
# =====
def create_array_from_list():
    """Plain English: hand NumPy a Python list, and it wraps it up as
    an array. Simple and explicit, but tedious for long sequences."""
    return np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

def create_array_with_arange():
    """Plain English: np.arange(start, stop) generates a sequence of
    numbers directly, without typing them all out. Note that `stop`
    (11) is EXCLUDED, which is why we write 11, not 10, to include 10."""
    return np.arange(1, 11)

# =====
# 2. INSPECT THE ARRAY'S BASIC PROPERTIES
#     Plain English: every NumPy array carries a few useful facts
#     about itself, which we can simply ask for.
# =====
def describe_array(arr):
    """Prints the array itself and its key attributes."""
    print(f"Array          : {arr}")
    print(f"Data type (dtype): {arr.dtype}")
    print(f"Shape           : {arr.shape}")
    print(f"Number of dims  : {arr.ndim}")
    print(f"Size (elements) : {arr.size}")

# =====
# 3. DEMO / DRIVER CODE
# =====
if __name__ == "__main__":

    # ---- Method 1: build the array from an explicit list ----
    arr1 = create_array_from_list()
    print("Array created with np.array([1, 2, ..., 10]):")
    describe_array(arr1)

    # ---- Method 2: build the same array with arange ----
    print()
    arr2 = create_array_with_arange()
    print("Array created with np.arange(1, 11):")
    describe_array(arr2)

    # ---- confirm both methods give the exact same result ----
    print()
    print("Are both arrays equal?", np.array_equal(arr1, arr2))

    # ---- BONUS: vectorized operations (no loop needed) ----
    print()
    print("Bonus: vectorized operations (applied to every element at once)")
    print(f"arr1 * 2          : {arr1 * 2}")
    print(f"arr1 + 100        : {arr1 + 100}")
    print(f"sum(arr1)         : {arr1.sum()}")
    print(f"mean(arr1)        : {arr1.mean()}")

    # ---- BONUS: indexing and slicing ----
```

```
print()
print("Bonus: indexing and slicing")
print(f"First element : {arr1[0]}")
print(f"Last element  : {arr1[-1]}")
print(f"First 5 values : {arr1[:5]}")
print(f"Every other   : {arr1[::2]}")
```

Output

```
Array created with np.array([1, 2, ..., 10]):  
Array          : [ 1  2  3  4  5  6  7  8  9 10]  
Data type (dtype): int64  
Shape          : (10,)  
Number of dims : 1  
Size (elements) : 10
```

```
Array created with np.arange(1, 11):  
Array          : [ 1  2  3  4  5  6  7  8  9 10]  
Data type (dtype): int64  
Shape          : (10,)  
Number of dims : 1  
Size (elements) : 10
```

Are both arrays equal? True

```
Bonus: vectorized operations (applied to every element at once)  
arr1 * 2       : [ 2  4  6  8 10 12 14 16 18 20]  
arr1 + 100     : [101 102 103 104 105 106 107 108 109 110]  
sum(arr1)      : 55  
mean(arr1)     : 5.5
```

```
Bonus: indexing and slicing  
First element  : 1  
Last element   : 10  
First 5 values : [1 2 3 4 5]  
Every other    : [1 3 5 7 9]
```

Output Screenshots (Online Python Compiler)

Screenshot 1

The screenshot shows the Online Python Compiler interface. The main editor displays a Python script titled 'main.py' with the following content:

```
1 """
2 =====
3 CREATING A ONE-DIMENSIONAL NUMPY ARRAY (THE NUMBERS 1 TO 10)
4 =====
5 IN PLAIN ENGLISH:
6 A NumPy array is like a Python list, but built specifically for
7 fast, whole-array math. A "one-dimensional" array is simply a
8 single row of numbers - no rows-and-columns grid, just one
9 straight line of values, like beads on a single string.
10
11 This script creates that array two different ways, inspects its
12 basic properties, and shows a few quick examples of why NumPy
13 arrays are so useful compared to plain Python lists.
14 =====
15 """
16
17 import numpy as np
18
19
20 # =====
21 # 1. CREATE THE ARRAY - TWO COMMON WAYS
22 # =====
23
24 def create_array_from_list():
25     """Plain English: hand NumPy a Python list, and it wraps it up as
26     an array. Simple and explicit, but tedious for long sequences."""
27     return np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

The right sidebar shows the output of the code execution:

```
data type (dtype):
int64
Shape
(10,)
Number of dims : 1
Size (elements) : 10

Array created with
np.arange(1, 11):
Array
1 2 3 4 5 6 7
8 9 10]
Data type (dtype):
int64
Shape
(10,)
Number of dims : 1
Size (elements) : 10

Are both arrays
equal? True

Bonus: vectorized
operations (applied
to every element at
once)
```

Screenshot 2

The screenshot shows the Online Python Compiler interface. The main editor displays a Python script titled 'main.py' with the following content:

```
1 """
2 =====
3 CREATING A ONE-DIMENSIONAL NUMPY ARRAY (THE NUMBERS 1 TO 10)
4 =====
5 IN PLAIN ENGLISH:
6 A NumPy array is like a Python list, but built specifically for
7 fast, whole-array math. A "one-dimensional" array is simply a
8 single row of numbers - no rows-and-columns grid, just one
9 straight line of values, like beads on a single string.
10
11 This script creates that array two different ways, inspects its
12 basic properties, and shows a few quick examples of why NumPy
13 arrays are so useful compared to plain Python lists.
14 =====
15 """
16
17 import numpy as np
18
19
20 # =====
21 # 1. CREATE THE ARRAY - TWO COMMON WAYS
22 # =====
23
24 def create_array_from_list():
25     """Plain English: hand NumPy a Python list, and it wraps it up as
26     an array. Simple and explicit, but tedious for long sequences."""
27     return np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

The right sidebar shows the output of the code execution:

```
data type (dtype):
int64
Shape
(10,)
Number of dims : 1
Size (elements) : 10

Array created with
np.arange(1, 11):
Array
1 2 3 4 5 6 7
8 9 10]
Data type (dtype):
int64
Shape
(10,)
Number of dims : 1
Size (elements) : 10

Are both arrays
equal? True

Bonus: vectorized
operations (applied
to every element at
once)
```

Screenshot 3

The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` includes:

```
59 # ---- Method 2: build the same array with arange ----
60 print()
61 arr2 = create_array_with_arange()
62 print("Array created with np.arange(1, 11):")
63 describe_array(arr2)
64
65 # ---- confirm both methods give the exact same result ----
66 print()
67 print("Are both arrays equal?", np.array_equal(arr1, arr2))
68
69 # ---- BONUS: vectorized operations (no loop needed) ----
70 print()
71 print("Bonus: vectorized operations (applied to every element at once)")
72 print(f"arr1 * 2      : {arr1 * 2}")
73 print(f"arr1 + 100    : {arr1 + 100}")
74 print(f"sum(arr1)      : {arr1.sum()}")
75 print(f"mean(arr1)    : {arr1.mean()}")
76
77 # ---- BONUS: indexing and slicing ----
78 print()
79 print("Bonus: indexing and slicing")
80 print(f"First element : {arr1[0]}")
81 print(f"Last element  : {arr1[-1]}")
82 print(f"First 5 values  : {arr1[:5]}")
83 print(f"Every other    : {arr1[::2]}")
```

The output on the right shows the array details and the results of the operations:

```
data type (dtype): int64
Shape: (10,)
Number of dims: 1
Size (elements): 10

Array created with np.arange(1, 11):
Array: [ 1  2  3  4  5  6  7  8  9 10]
Data type (dtype): int64
Shape: (10,)
Number of dims: 1
Size (elements): 10

Are both arrays equal? True

Bonus: vectorized operations (applied to every element at once)
```

Screenshot 4

This screenshot shows the same code as Screenshot 4, but with the execution output visible on the right side. The output includes:

```
Array created with np.array([1, 2, ..., 10]):
Array: [ 1  2  3  4  5  6  7  8  9 10]
Data type (dtype): int64
Shape: (10,)
Number of dims: 1
Size (elements): 10

Array created with np.arange(1, 11):
Array: [ 1  2  3  4  5  6  7  8  9 10]
Data type (dtype): int64
Shape: (10,)
Number of dims: 1
Size (elements): 10

Are both arrays equal? True

Bonus: vectorized operations (applied to every element at once)
arr1 * 2      : [ 2  4  6  8 10 12 14 16 18 20]
arr1 + 100    : [101 102 103 104 105 106 107 108 109 110]
sum(arr1)     : 55
mean(arr1)    : 5.5

Bonus: indexing and slicing
First element : 1
Last element  : 10
First 5 values : [ 1  2  3  4  5]
Every other    : [ 1  3  5  7  9]
```

Screenshot 5

The screenshot shows the Programiz Online Python Compiler interface. The code editor on the left contains a Python script named `main.py`. The script defines a `describe_array` function that prints the array itself and its key attributes (dtype, shape, number of dimensions, and size). It then creates an array `arr1` using `np.arange(1, 11)` and prints it. The output panel on the right shows the results of running the code. It displays the data type (`int64`), shape (`(10,)`), number of dimensions (`1`), and size (`10`). It also shows the array created with `np.arange(1, 11)` as `[1 2 3 4 5 6 7 8 9 10]`. The output confirms that the array is a one-dimensional array of integers with 10 elements.

```
main.py
11 # (11) is EXCLUDED, which is why we write 11, not 10, to include 10.
33 return np.arange(1, 11)
34
35
36 # -----
37 # 2. INSPECT THE ARRAY'S BASIC PROPERTIES
38 # Plain English: every NumPy array carries a few useful facts
39 # about itself, which we can simply ask for.
40 # -----
41 def describe_array(arr):
42     """Prints the array itself and its key attributes."""
43     print(f"Array : {arr}")
44     print(f"Data type (dtype): {arr.dtype}")
45     print(f"Shape : {arr.shape}")
46     print(f"Number of dims : {arr.ndim}")
47     print(f"Size (elements) : {arr.size}")
48
49
50 # -----
51 # 3. DEMO / DRIVER CODE
52 # -----
53 if __name__ == "__main__":
54
55     # ---- Method 1: build the array from an explicit list ----
56     arr1 = create_array_from_list()
57     print("Array created with np.arange(1, 2, ..., 101):")
```

Output

```
Data type (dtype):
int64
Shape :
(10,)
Number of dims : 1
Size (elements) : 10

Array created with
np.arange(1, 11):
Array : [
1 2 3 4 5 6 7
8 9 10]
Data type (dtype):
int64
Shape :
(10,)
Number of dims : 1
Size (elements) : 10

Are both arrays
equal? True

Bonus: vectorized
operations (applied
to every element at
once)
```

Screenshot 6

The screenshot shows the Programiz Online Python Compiler interface. The code editor on the left contains a Python script named `mc`. The script defines a `describe_array` function that prints the array itself and its key attributes (dtype, shape, number of dimensions, and size). It then creates an array `arr1` using `np.arange(1, 11)` and prints it. The output panel on the right shows the results of running the code. It displays the data type (`int64`), shape (`(10,)`), number of dimensions (`1`), and size (`10`). It also shows the array created with `np.arange(1, 11)` as `[1 2 3 4 5 6 7 8 9 10]`. The output confirms that the array is a one-dimensional array of integers with 10 elements.

```
mc
59 # ----
60 # Method 2: build
the same array
with arange ----
61 print()
62 arr2 =
create_array_with
_arange()
63 print("Array
created with
np.arange(1,
11):")
64 describe_array(ar
r2)
65
66 # ----
confirm both
methods give the
exact same
result -----
67 print()
68 print("Are
```

Output

```
Data type (dtype): int64
Shape : (10,)
Number of dims : 1
Size (elements) : 10

Array created with np.arange(1, 11):
Array : [1 2 3 4 5 6 7 8 9 10]
Data type (dtype): int64
Shape : (10,)
Number of dims : 1
Size (elements) : 10

Are both arrays equal? True

Bonus: vectorized operations (applied to every element at once)
arr1 * 2 : [2 4 6 8 10 12 14 16 18 20]
arr1 + 100 : [101 102 103 104 105 106 107 108 109 110]
sum(arr1) : 55
mean(arr1) : 5.5

Bonus: indexing and slicing
First element : 1
Last element : 10
First 5 values : [1 2 3 4 5]
Every other : [1 3 5 7 9]
```

Result

Both methods produced identical one-dimensional arrays of shape (10,), dtype int64, confirming that `np.array()` and `np.arange(1, 11)` are equivalent for this task.